USD8 Shapley Fee-Routing Specification

Status: design specification, ready for implementation upon Cover Pool contract surface. **Source**: ports VibeSwap's production-deployed Shapley distribution mechanism (`contracts/incentives/ShapleyDistributor.sol`, ~1100 LOC, 82 BatchMath tests passing) to USD8's Cover Pool surface. **Audience**: USD8 protocol team. Treat this as a porting brief, not a from-scratch design. The math, the axioms, and the on-chain computation pattern are battle-tested in VibeSwap; this document maps them onto USD8's specific architecture.

What this document is

USD8's Cover Pool is structurally a cooperative game. Multiple liquidity providers contribute capital that underwrites coverage for USD8 holders. When fees accrue from yield deployment, they need to be distributed back to those providers in proportion to each provider's contribution to the system's stability. The question this document answers is: *what does "in proportion to contribution" actually mean, formally, in a way that survives audit?*

The answer that has stood for seventy-three years is the Shapley value (1953). It is the unique allocation rule that simultaneously satisfies efficiency, symmetry, null-player neutrality, and additivity — the four properties that any defensible fairness mechanism must have. No other allocation rule has all four.

What follows is the specification for porting VibeSwap's working Shapley implementation to USD8. Five components transfer with minor reframing. One component drops because it requires a market signal USD8 doesn't have. The total integration effort is bounded; the substrate-independence of the math is the load-bearing property that makes the port possible.

Section I — Why Shapley, formally

A cover pool with n liquidity providers is a cooperative game (N, v) where $N = \{1, 2, ..., n\}$ is the set of providers and $v : 2^N \to \mathbb{R}_{\geq 0}$ is the value function — the total fees the system can produce given the participation of any subset $S \subseteq N$.

For each provider i, the Shapley value $\phi_i(v)$ is defined as their marginal contribution averaged across all possible orderings of arrival:

\[*i(v)* = {S N {i}} ,\]

Lloyd Shapley proved in 1953 that this is the *unique* allocation satisfying four properties simultaneously:

- **Efficiency**: $\sum_i \phi_i(v) = v(N)$ — the total payout equals the total value generated.
- **Symmetry**: if i and j contribute identically to every coalition, then $\phi_i = \phi_j$.
- **Null-player neutrality**: if i adds zero value to every coalition, then $\phi_i = 0$.
- **Additivity**: if two games v and w are summed, the Shapley value of the combined game equals the sum of the individual Shapley values.

These four properties exhaust the space of "fairness" — any allocation rule satisfying all four is necessarily the Shapley value. This is a theorem, not a stylistic preference. The Cover Pool inherits a defensible fairness foundation simply by adopting it.

The on-chain computability problem

The Shapley value as defined above is not directly computable on-chain. The sum is over all 2^{n-1} subsets of $N \setminus \{i\}$. For 100 providers, that is approximately 6.3×10^{29} terms per provider. No blockchain can do this.

The substrate-aware solution

The trick that makes Shapley tractable on-chain is to choose a *linear* characteristic function. If $v(S) = \sum_{i \in S} w_i$ for some per-participant weight w_i, then the Shapley value collapses to the closed form:

\[_i(v) = w_i \]

This is exact. No approximation. All four axioms hold exactly under this restriction. The on-chain cost drops from exponential to linear in n.

The remaining design question — and where most of the engineering goes — is *what to put in the weight* w_i. A naive "weight = capital deposited" recovers proportional-by-capital distribution, which is what every existing protocol does, and which fails to reward providers for the multiple dimensions in which they actually contribute. The richer the weight function, the more accurately it captures real contribution; the more complex, the harder to audit. The five-component weight in the next section is what VibeSwap arrived at after working the problem.

Section II — The five-component weight

Each provider's weight w_i is computed from five components. Four are additive contributions; one is a quality multiplier; one is an optional pioneer bonus. The computation is implemented in `ShapleyDistributor.sol:893-952` and runs in O(n) on-chain.

The weight formula is:

\[w_i = (d_i + t_i + s_i + r_i) q_i p_i \]

where each term is described below. The four base weights sum to 1.00; the multiplications by q_i and p_i are applied last.

Component 1 — Direct Contribution (d_i, weight 40%)

What it measures: the raw capital committed to the Cover Pool by provider i.

Formula: d_i = capital$_i$, normalized by the total Cover Pool capital. Linear in deposit size. A provider committing 100 ETH of coverage capacity has 10× the direct score of a provider committing 10 ETH.

Rationale: someone has to put capital at risk. This is the floor. No matter how clever the other dimensions are, a provider who never deposited capital cannot earn fees from this component.

USD8 mapping: directly. The Cover Pool already tracks per-provider capital in the existing vault accounting. No new measurement needed.

Component 2 — Enabling Contribution (t_i, weight 30%)

What it measures: the *duration* during which provider i's capital was available to underwrite coverage. A provider whose capital sat in the pool for 200 days enabled 200 days of coverage availability; a provider whose capital arrived yesterday enabled yesterday's.

Formula: logarithmic in tenure.

\[t_i = \]

A 1-day tenure scores 1.0 (minimal credit for showing up); a 7-day tenure scores ~1.9; a 30-day tenure ~2.7; a 365-day tenure ~4.2. The diminishing-returns curve is intentional: the first weeks of a Cover Pool's existence matter disproportionately for trust formation. The hundredth week matters less.

Rationale: capital that arrived early did harder work. A provider who deposited when the pool had no track record was taking on uncertainty that a provider arriving after a year of clean operation was not. The

enabling component pays them back for that.

USD8 mapping: directly, with one input adjustment. VibeSwap measures `tenure = current_time - deposit_time`. USD8 should measure `tenure = min(withdrawal_time, current_time) - deposit_time` — i.e., actual capital deployment duration, not just nominal time-since-deposit. This change is necessary because USD8's 14-day cooldown on withdrawals means a provider who initiated withdrawal 14 days ago is no longer "in the pool" for tenure purposes from withdrawal-initiation onward.

Component 3 — Scarcity Contribution (s_i, weight 20%)

What it measures: in VibeSwap, this rewards providers who supply liquidity on the *scarce* side of an order imbalance — e.g., sell-side liquidity in an 80/20 buy-skewed batch.

USD8 mapping — DROPS by default. A cover pool does not have natural sides. There is no "scarce" coverage class unless USD8 explicitly defines one (e.g., "smart-contract risk coverage is scarce relative to price-risk coverage"). Without a market-driven imbalance signal, scarcity rewards become a governance decision wearing the mask of mechanism, which violates the Augmented Mechanism Design principle that fairness should not be a discretionary choice.

Optional reinterpretation: if USD8 wants to use this component, redefine "scarce" as "underrepresented coverage class" — e.g., if smart-contract risk underwriters are 20% of the pool but cover 80% of demand, give them a scarcity multiplier. This requires per-coverage-class accounting and a defensible definition of the scarcity threshold. Recommended only if there is genuine market demand asymmetry across coverage classes.

Default recommendation: drop. The 20% weight redistributes to Direct (lifted from 40% to 50%) and Enabling (lifted from 30% to 40%), preserving the additive sum to 1.00 over the four remaining components. Stability holds at 10%.

Component 4 — Stability Contribution (r_i, weight 10%, lifted to 10% if Scarcity drops)

What it measures: a provider who does not panic-withdraw during a stress event has provided real value beyond what their capital alone reflects. They held the pool's continuity together when others were pulling capital out.

Formula: r_i is an externally-supplied score in $[0, 1]$ representing the provider's behavior during recent stress events. It is computed off-chain (because identifying "stress events" requires interpretation that smart contracts cannot do reliably) and passed in at game creation.

Rationale: providers who exit during a claim surge or a yield-strategy migration window create exit pressure that compounds the original stress. Providers who hold contribute liquidity at exactly the moment liquidity is most needed. The Shapley distribution should reward this asymmetry.

USD8 mapping: directly, with input redefinition. VibeSwap's stress signal is price volatility on the underlying AMM. USD8's stress signal should be a function of: - Active claims pressure (claims in the 10-day window relative to historical baseline) - Cover Pool withdrawal rate during high-claims periods - Yield-strategy migration windows (a claim-vulnerable period for the pool)

The off-chain keeper that computes the stability score for each provider needs access to these three time series. USD8's existing keeper infrastructure (already used to compute Cover Scores) is the natural home for this computation.

Component 5 — Quality Multiplier (q_i, range 0.5×–1.5×)

What it measures: a Sybil-resistant reputation multiplier that rewards historical good behavior across three orthogonal dimensions:

\[q_i = 0.5 + (_i + _i + _i) \]

where each of the three input scores is in [0, 10000] basis points. A new address with no history scores at the floor (0.5×); a maximally-positive participant scores at the ceiling (1.5×).

Rationale: prevents Sybil attacks. A provider splitting one address into ten loses the quality history on each fragment, capping their multiplier at 0.5× per fragment. Aggregating ten fragments at 0.5× yields 5×, while staying as one address scoring 1.5× yields 1.5× — but the latter applies to *all* the capital, while the former requires fragmenting capital across ten addresses, which loses the absolute size advantage. The math discourages Sybil aggressively.

USD8 mapping: directly. The three dimensions translate cleanly: - **Activity**: how frequently has the provider participated in governance, adjusted positions, or interacted with the protocol over the rolling window? - **Reputation**: do prior claims involving this provider's coverage have a history of clean resolution, or is there a pattern of disputes? - **Economic**: how large and how diverse is the provider's historical contribution across coverage classes and tenure periods?

Quality scores are snapshotted at the start of each fee-distribution game to prevent front-running between game creation and reward claim — a point worth preserving in the USD8 implementation.

Optional Component — Pioneer Bonus (p_i, range 1.0×–2.0×)

What it measures: an early-participant bonus to incentivize bootstrapping the pool when it is small and risky.

Formula: $p_i = 1.0 + \min(\text{pioneerScore}_i/20000, 1.0)$, capped at 2.0×.

USD8 mapping: optional. If USD8 has other bootstrap incentives (e.g., the Booster NFTs that already exist), this component can be omitted. If USD8 wants additional early-LP weighting beyond what Booster NFTs provide, the pioneer multiplier is the right surface — and it composes cleanly with everything else because it's a pure final multiplier.

Section III — The fee flow

This section describes the call sequence from yield-strategy revenue to LP wallets. It mirrors VibeSwap's deployed flow with substitutions for USD8's specific contracts. Where USD8 contract names are not yet known, placeholders (in italics) indicate the integration point.

```
1. USDC yield strategy → revenue accrual in *USD8 protocol vault*
2. *USD8 protocol vault* → forwardFees(USDC) → ShapleyDistributor
3. *USD8 keeper* → createGame(gameId, totalFees, USDC, [providers])
   - providers[] includes per-provider: capital, tenure, stability
score, quality scores
4. anyone → ShapleyDistributor.computeShapleyValues(gameId)
   - applies the 5-weight formula
   - enforces the Lawson Floor (no provider receives less than the dust
threshold)
   - enforces efficiency (last-participant remainder eliminates rounding
loss)
5. each LP → ShapleyDistributor.claimReward(gameId)
   - direct transfer of USDC to msg.sender
```

Steps 4 and 5 are permissionless. Step 3 is keeper-driven (any authorized address can create the game, with the per-game inputs assembled from the off-chain measurements). Step 2 is admin-keyed today in VibeSwap, but USD8 can choose to make it permissionless (anyone can poke the protocol vault to forward accumulated fees) or keep it keyed depending on operational preference.

The full implementation is at `ShapleyDistributor.sol:463-878`. Key fairness enforcement details:

- **Lawson Floor** (`:709-743`): a small-floor mechanism ensuring every provider with non-zero contribution receives at least the dust threshold. Prevents the "rounded to zero" failure mode where micro-contributors are pushed below the gas-wasteful payout threshold and lose their share entirely.

- **Efficiency correction** (`:746-756`): the last participant in the distribution receives any rounding remainder so $\sum_i \phi_i = v(N)$ holds exactly. Without this, integer division inevitably leaks dust.
- **Pairwise Proportionality auditing** (`PairwiseFairness.sol`): a post-hoc verification function that confirms the ratio-preservation property $\phi_i/\phi_j = w_i/w_j$ holds within rounding tolerance. Run off-chain or on-chain as a sanity check.

Section IV — What USD8 needs to provide

To integrate this mechanism, the following are required from USD8's existing or near-future contract surface:

Required from on-chain state

1. **Per-provider Cover Pool capital** (current and historical). The existing Cover Pool already tracks per-LP capital. We need a way to query each provider's capital at game-creation time.

2. **Per-provider deposit timestamp**. Required for the Enabling component. If the existing Cover Pool stores deposit-time per LP (which it should for the 14-day cooldown anyway), this is already available.

3. **Withdrawal-initiated timestamp** (per provider). Required for Enabling component's adjusted tenure formula. Likely already tracked because of the cooldown mechanism.

4. **Cover Pool fee-token balance**. The amount of accrued fees ready to distribute. Each game's total value $v(N)$ is the fee-token balance transferred to ShapleyDistributor.

Required from off-chain measurement

5. **Stability score** (per provider, per game, $[0, 10000]$ bps). Computed by USD8's keeper from claims-pressure and withdrawal-rate time series. Submitted at game creation.

6. **Quality scores** (three dimensions, per provider, [0, 10000] bps each). Activity, reputation, economic. Computed by USD8's reputation keeper. Updated per epoch via authorized controller.
7. **Pioneer score** (per provider, [0, 10000] bps, optional). Only needed if pioneer bonus is enabled.

Brevis integration

USD8's existing partnership with Brevis (ZK Coprocessor for Cover Score computation) is the natural extension point for the off-chain inputs above. Stability scores and quality scores can be computed by Brevis and submitted with cryptographic proofs of correct computation, preserving the Walkaway Test commitment articulated in the existing Cover Pool documentation.

This is a meaningful design upgrade. VibeSwap's current implementation trusts the keeper to submit honest stability and quality scores. USD8, with Brevis already in the stack, can verify those scores on-chain. The Shapley distribution becomes simultaneously: - Game-theoretically fair (Shapley axioms) - Cryptographically verifiable (Brevis proofs over the input scores) - Walkaway-test resilient (functions if the team disappears)

We are not aware of another insurance protocol with all three properties.

Section V — Battle-testing

The mechanism described in this document is the deployed implementation of `contracts/incentives/ShapleyDistributor.sol` in the VibeSwap protocol.

- ~1100 lines of contract code
- 82 BatchMath tests passing
- The five-axiom fairness verification (Efficiency, Symmetry, Null Player, Pairwise Proportionality, Time Neutrality) is executed by `PairwiseFairness.sol` and exercised in the test suite

- Halving and time-neutrality logic is differentiated by game type (TOKEN_EMISSION uses halving; FEE_DISTRIBUTION does not), allowing USD8 to choose the appropriate mode for fee distribution (FEE_DISTRIBUTION is correct — the goal is to distribute the realized fee pool fairly, not to apply Bitcoin-style emission scheduling)
- The contract has an on-chain Pairwise Proportionality verifier that can be queried at any time to confirm fairness invariants hold

This is not an experiment. It is a mechanism that has run, been tested, and continues to function in production. USD8's adoption of it inherits seven decades of game-theoretic foundation, plus the engineering work already done to make it computable on a real blockchain.

Section VI — Open questions for the USD8 team

Before implementation can begin, the following decisions need input from the USD8 protocol team:

1. **Cover Pool internals**. We need read access to: per-provider capital balances, deposit timestamps, and withdrawal-initiation timestamps. Are these already in the Cover Pool contract storage, or will they need to be added? If the Cover Pool is being designed now, this is the moment to ensure these are tracked.

2. **Scarcity component — drop or define?** Default recommendation is to drop, with the 20% weight redistributing to Direct (50%) and Enabling (40%). If USD8 has natural coverage-class asymmetry that could feed a defensible scarcity signal, we keep it; if not, drop is cleaner.

3. **Pioneer Bonus — keep or drop?** USD8 already has Booster NFTs as a holder-side bootstrap incentive. Whether to add a provider-side pioneer bonus on top depends on whether the Booster system is intended to cover both sides or just the holder side.

4. **Stability score input source**. Will USD8's existing keeper compute the stability scores, or should we propose a Brevis circuit for verifiable computation? The latter is a meaningful security upgrade; the former is faster to ship.

5. **Game cadence**. VibeSwap creates a new Shapley game per fee-distribution cycle (roughly per batch). What's USD8's intended fee-distribution cadence? Daily? Weekly? Per-claim-event? The cadence affects keeper economics and gas-cost amortization.

6. **Governance scope**. The four-weight formula has fixed coefficients (40/30/20/10 in VibeSwap, 50/40/10 if Scarcity is dropped). Should these be governance-tunable, or fixed in code? VibeSwap defaults to fixed (the coefficients are part of the math); USD8 may have different tradeoffs given its insurance-pool semantics.

7. **Anti-gaming concerns**. Are there USD8-specific attack vectors we should design against (e.g., capital ping-ponging in and out to game the tenure component)? VibeSwap's tenure formula is logarithmic specifically to make ping-ponging unprofitable, but the optimal anti-gaming tuning is protocol-specific.

These are decision-points, not blockers. Once answered, the implementation surface is bounded and the work is largely a port-and-adapt rather than a clean-room build.

Appendix A — Why not just proportional-by-capital?

The simplest possible fee distribution is "fees proportional to capital deposited." This is what most insurance protocols and AMMs use. It has the appeal of simplicity and the disadvantage of being incomplete.

Proportional-by-capital pays a provider who deposited yesterday at the same rate per unit of capital as a provider who deposited a year ago — even though the year-ago provider underwrote uncertainty during the system's most fragile period. Proportional-by-capital pays a provider who

panic-withdraws during a claim surge nothing differently from a provider who held the line — even though the latter actively kept the pool functional. Proportional-by-capital pays a Sybil-fragmented attacker exactly the same as a unified large LP — even though the Sybil is gaming the system.

Each of these is a real gap. Each maps cleanly to one of the additional components in the five-weight formula. The complexity is not arbitrary; it is the minimum complexity required to close known fairness gaps.

The Shapley value is the unique fair allocation. The five-weight formula is the engineering compromise that makes Shapley computable on-chain while preserving its fairness properties. Together they are the closest a real-world insurance pool can get to "every provider receives exactly what they contributed, no more and no less" — which is, in the end, the only defensible answer.

Appendix B — Comparison with naive Shapley

A common objection: "Shapley is exponential. You can't do it on-chain."

This is true for the *general* Shapley value, where the characteristic function $v(S)$ can be arbitrary. The exponential cost comes from needing to evaluate $v(S)$ for each of 2^n subsets.

But once we restrict to *linear* characteristic functions $v(S) = \sum_{i \in S} w_i$, the Shapley value collapses to the closed form $\phi_i = w_i \cdot v(N)/\sum_j w_j$. This is exact, not approximate. All four Shapley axioms hold under this restriction. Computation is O(n).

The trick is therefore to put the engineering effort into the per-participant weight w_i (which is what the five-component formula does) rather than into the cross-coalition value function (which is what the exponential formulation requires).

VibeSwap discovered this empirically. USD8 inherits the result.

Specification authored by William Glynn with primitive-assist from JARVIS. Source implementation: `vibeswap/contracts/incentives/ShapleyDistributor.sol` *(production, 82 BatchMath tests passing). Open to review, refinement, and reframing as Rick's team determines what fits their architecture. Implementation will commence upon access to USD8 Cover Pool contract internals.*